

Fitting and validating

Every model in scikit-learn has the same three verbs, and almost every mistake is about which data they were called on.

The three verbs

`model.fit(X_train, y_train)`

Learn from the training data. Only ever call this on training data.

`model.predict(X_new)`

Apply what was learned to rows the model has not seen.

`transformer.transform(X)`

Apply a fitted step. `fit_transform` on the training split only.

THE RULE THE WHOLE BLOCK RESTS ON

Anything learned from data must be learned from the training split alone.

That includes what does not feel like learning: the mean used to fill missing values, a scaler's range, the categories an encoder knows.

Inside a Pipeline the rule is enforced structurally, instead of depending on you remembering it at the right moment.

The pipeline skeleton

```
pre = ColumnTransformer([
    ('num', SimpleImputer(strategy='median'), NUMERIC),
    ('cat', OneHotEncoder(handle_unknown='ignore',
                           sparse_output=False), CATEGORICAL),
])

pipe = Pipeline([('pre', pre), ('model', Ridge())])
pipe.fit(X_train, y_train)
```

One object that preprocesses and predicts. Cross-validation then refits the whole thing inside every fold, which is what keeps the estimate honest.

Splitting, when rows are not independent

`train_test_split(X, y)`

A random split. Correct only when every row is independent of every other row.

`GroupShuffleSplit(...)`

One split, keeping each group wholly on one side. Pass `groups=`.

`GroupKFold(n_splits=5)`

Cross-validation that never puts the same group in train and validation.

`cross_val_score(pipe, X, y,
 cv=cv, groups=g)`

One score per fold. Look at all of them, not only the mean.

ASK BEFORE YOU SPLIT

Does one real-world thing produce several rows here?

One patient with several admissions, one hospital with many patients, one host with many listings. If so, a random split puts that thing on both sides and the score you get back is too good.

Group by whatever the model will not have seen before at the moment it is used.

Models and metrics

What each family assumes, and what each number means. Which metric to report is a decision about which mistake costs more, not a technical default.

The families, and the knob that matters most

<code>LinearRegression()</code>	A weighted sum. Readable coefficients, none of them causal.
<code>Ridge(alpha=1.0)</code>	Linear, large coefficients penalised. Higher alpha, simpler fit.
<code>LogisticRegression(C=1.0)</code>	Linear boundary, outputs a probability. Smaller C penalises more.
<code>KNeighborsClassifier(k)</code>	Vote of the k nearest rows. Needs scaling; suffers in high dimension.
<code>DecisionTreeClassifier(max_depth=5)</code>	Nested yes or no questions. Unbounded depth memorises rows.
<code>RandomForestClassifier(n_estimators=300)</code>	Many decorrelated trees, averaged. More trees only costs time.
<code>HistGradientBoosting*(learning_rate, max_iter)</code>	Trees fitted in sequence on the error. Can overfit as trees are added.

Regression metrics

MAE	Mean absolute error. In the units of the target, and it treats every euro of error alike.
RMSE	Same units, but large errors are penalised far more heavily.
R squared	Share of variance explained. Unitless, not comparable across targets.

Classification metrics

All of them are computed from the four cells of the confusion matrix: true and false, positive and negative.

accuracy	Share of predictions right. Depends on how common the positive is.
precision	Of those flagged, how many were positive. The cost of a false alarm.
recall	Of the real positives, how many we caught. The cost of a miss.
f1	Harmonic mean of precision and recall. One number, and it hides which of the two is weak.
roc_auc	Ranking quality across all thresholds. Silent about any one of them.

TWO QUESTIONS TO ASK OF ANY SCORE

What does the do-nothing model get, and how much does this number move between folds?

Score DummyRegressor or DummyClassifier on the same data first: a metric with no floor attached cannot be interpreted. Then report the spread across folds, not the mean alone, because two models whose fold ranges overlap have not been shown to differ.